CertificateAuthority

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring. 4 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-CertificateAuthority-go.

CertificateAuthority - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/CertificateAuthority. Entry point: ./cmd/certificateauthority. Module: arrowhead/ah5-support-go/certificateauthority. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8448 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8448"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
CertificateAuthority	Role/profile-constrained CSR signing, chain checks, certificate revocation, signed CRL export and trusted keys	Java SQL import, automatic CRL-file publication, wider relying-party revocation enforcement, HSM/OCSP/HA

Detailed implementation evidence

1 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-CertificateAuthority-rust.

CertificateAuthority (Rust)

Arrowhead 5.2.2 CertificateAuthority support system. Entry point src/main.rs is built as binary certificateauthority by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/certificateauthority.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/CertificateAuthority, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin certificateauthority -- <config.json>

Listening port

Default 127.0.0.1:8448 (Arrowhead default, role certificateauthority). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
CertificateAuthority	Native CSR verification/signing, profiles, revocation, PEM CRLs, trusted keys	Explicit SANs only (no CSR extension copying); automatic key renewal/root rollover, OCSP/HSM/HA, full validity/DTO normalization

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data

migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring/documentation/v5.2.2/CertificateAuthority-v5.2.2-certificateAuthorityManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring/documentation/v5.2.2/CertificateAuthority-v5.2.2-certificateSigning-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring/documentation/v5.2.2/CertificateAuthority-v5.2.2-certificateValidation-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-CertificateAuthority-java-spring/documentation/v5.2.2/CertificateAuthority-v5.2.2-sysMonitor-AMQP-IDD.tex

- Go: Eclipse-Arrowhead-go/ah5-CertificateAuthority-go/documentation/CertificateAuthority/CertificateAuthority-HTTPS-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/CertificateAuthority/CertificateAuthority-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

certificateAuthorityManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"certificateAuthorityManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `certificateAuthorityManagement` service of the CertificateAuthority system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation list-certificates . . . 4
2.2 operation revoke-certificate . . . 4
2.3 operation revoke-certificate-by-content . . . 5
2.4 operation list-keys . . . 5
2.5 operation add-key . . . 6
2.6 operation delete-key . . . 6

3 Data Models **7**
3.1 struct CertificateRecord . . . 7
3.2 struct RemovalResult . . . 7
3.3 struct RevocationRequest . . . 7
3.4 struct RevocationResult . . . 7
3.5 struct TrustedKeyRecord . . . 7
3.6 struct TrustedKeyRequest . . . 8
3.7 struct TrustedKeyResult . . . 8
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `certificateAuthorityManagement` service interface of the CertificateAuthority system, which provides operator management of the certificate register and of trusted keys, including revocation.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `certificateAuthorityManagement` service interface

This document provides the Interface Design Description IDD to the *certificateAuthorityManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `certificateAuthorityManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `certificate-authority`, port 8448 and base path `/certificate-authority`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`list-certificates`	GET	`/mgmt/certificates`
`revoke-certificate`	DELETE	`/mgmt/certificates/{id}`
`revoke-certificate-by-content`	POST	`/mgmt/revocations`
`list-keys`	GET	`/mgmt/keys`
`add-key`	PUT	`/mgmt/keys`
`delete-key`	DELETE	`/mgmt/keys/{id}`

2 Service Operations

The interfaces of the `certificateAuthorityManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `certificateAuthorityManagement` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 GET /certificate-authority/mgmt/certificates

Operation: list-certificates
Output: Array<CertificateRecord>

Returns the certificate register, newest first.

A successful call returns `200 OK` with the output in the body.

```
GET /certificate-authority/mgmt/certificates HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An list-certificates invocation.

```
[
  {
    "id": 42,
    "serial_number": "4189327123456789",
    "common_name": "Sensor1.TestCloud.Company.arrowhead.eu",
    "created_by": "Onboarding",
    "valid_from": "2026-10-22T12:00:00Z",
    "valid_until": "2027-10-22T12:00:00Z",
    "certificate_pem": "-----BEGIN CERTIFICATE-----\\n..."
  }
]
```

Listing 2: An list-certificates response.

2.2 DELETE /certificate-authority/mgmt/certificates/*{id}*

Operation: revoke-certificate
Input: String
Output: RemovalResult

Revokes a register record by identifier (`{id}`) and republishes the CRL.

Path parameter: *id*.

A successful call returns `200 OK` with the output in the body.

```
DELETE /certificate-authority/mgmt/certificates/<id> HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An revoke-certificate invocation.

```
{
  "success": true
}
```

Listing 4: An revoke-certificate response.

2.3 POST /certificate-authority/mgmt/revocations

Operation: revoke-certificate-by-content
Input: RevocationRequest
Output: RevocationResult

Revokes any certificate of this CA given by content; unknown ones are added to the register as revoked.
A successful call returns `200 OK` with the output in the body.

```
POST /certificate-authority/mgmt/revocations HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "certificate": "<PEM or Base64 DER>"
}
```

Listing 5: An revoke-certificate-by-content invocation.

```
{
  "id": 42,
  "serialNumber": "4189327123456789",
  "revoked": true,
  "changed": true
}
```

Listing 6: An revoke-certificate-by-content response.

2.4 GET /certificate-authority/mgmt/keys

Operation: list-keys
Output: Array<TrustedKeyRecord>

Returns the trusted keys.
A successful call returns `200 OK` with the output in the body.

```
GET /certificate-authority/mgmt/keys HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 7: An list-keys invocation.

```
[
  {
    "id": 42,
    "public_key_pem": "-----BEGIN PUBLIC KEY-----\\n...",
    "description": "Display may read",
    "valid_after": "string",
    "valid_before": "string",
    "created_at": "string"
  }
```

```
]
```

Listing 8: An list-keys response.

2.5 PUT /certificate-authority/mgmt/keys

Operation: add-key
Input: TrustedKeyRequest
Output: TrustedKeyResult

Adds a trusted key with validity.
A successful call returns `201 Created` with the output in the body.

```
PUT /certificate-authority/mgmt/keys HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "publicKey": "-----BEGIN PUBLIC KEY-----\\n...",
  "description": "Display may read",
  "validAfter": "2026-10-22T12:00:00Z",
  "validBefore": "2026-10-22T12:00:00Z"
}
```

Listing 9: An add-key invocation.

```
{
  "id": 42,
  "trusted": true
}
```

Listing 10: An add-key response.

2.6 DELETE /certificate-authority/mgmt/keys/*{id}*

Operation: delete-key
Input: String
Output: RemovalResult

Deletes a trusted key by identifier (`{id}`).
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
DELETE /certificate-authority/mgmt/keys/<id> HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 11: An delete-key invocation.

```
{
  "success": true
}
```

Listing 12: An delete-key response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.8, which are used to represent things like hashes, identifiers and texts.

3.1 struct CertificateRecord

Object Field	Value Type	Description
"id"	Number	Record identifier.
"serial_number"	String	Serial number.
"common_name"	String	Subject common name.
"created_by"	String	Requesting system (`revoked-by:<caller>` for imported revocations).
"valid_from"	String	Start of validity.
"valid_until"	String	End of validity.
"revoked_at"	String	Revocation time, null if not revoked.
"certificate_pem"	String	The certificate, PEM.

3.2 struct RemovalResult

Object Field	Value Type	Description
"success"	Boolean	Whether a record was changed; 404 when false.

3.3 struct RevocationRequest

Object Field	Value Type	Description
"certificate"	String	Certificate to revoke, PEM or Base64 DER; must be issued by this cloud CA.

3.4 struct RevocationResult

Object Field	Value Type	Description
"id"	Number	Register record of the certificate.
"serialNumber"	String	Serial number.
"revoked"	Boolean	Always true.
"changed"	Boolean	False if it was already revoked.

3.5 struct TrustedKeyRecord

Object Field	Value Type	Description
"id"	Number	Record identifier.
"public_key_pem"	String	The key, PEM.
"description"	String	Description.
"valid_after"	String	Start of validity.
"valid_before"	String	End of validity.
"created_at"	String	Creation time.

3.6 struct TrustedKeyRequest

Object Field	Value Type	Description
"publicKey"	String	PEM SubjectPublicKeyInfo (alias `publicKeyPem`).
"description"	String	Description.
"validAfter"	DateTime	Start of validity; default now.
"validBefore"	DateTime	End of validity; default now plus `CERT_VALIDITY_DAYS`.

3.7 struct TrustedKeyResult

Object Field	Value Type	Description
"id"	Number	Record identifier.
"trusted"	Boolean	Always true.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “certificateAuthorityManagement – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateAuthorityManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

certificateSigning HTTP/TLS/JSON

Interface Design Description

Service ID: *"certificateSigning"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `certificateSigning` service of the CertificateAuthority system.

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation sign 4

3 Data Models **5**
3.1 struct CertificateSigningRequest 5
3.2 struct CertificateSigningResult 5
3.3 Primitives 5

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the `certificateSigning` service interface of the CertificateAuthority system, which provides signing of proof-of-possession certificate requests with the local-cloud CA, under the Arrowhead certificate profiles.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `certificateSigning` service interface

This document provides the Interface Design Description IDD to the *certificateSigning – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `certificateSigning` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `certificate-authority`, port 8448 and base path `/certificate-authority`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`sign`	POST	`/sign`

2 Service Operations

The interfaces of the `certificateSigning` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `certificateSigning` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 POST /certificate-authority/sign

Operation: sign
Input: CertificateSigningRequest
Output: CertificateSigningResult

Signs a CSR for the requester's permitted profile and records the certificate.

A successful call returns `201 Created` with the output in the body.

```
POST /certificate-authority/sign HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "encodedCSR": "-----BEGIN CERTIFICATE REQUEST-----\\n...",
  "profile": "sy",
  "addresses": [
    "sensor1"
  ],
  "validAfter": "2026-10-22T12:00:00Z",
  "validBefore": "2026-10-22T12:00:00Z"
}
```

Listing 1: An sign invocation.

```
{
  "id": 42,
  "certificateChain": [
    "<leaf Base64 DER>",
    "<CA Base64 DER>"
  ],
  "commonName": "Device1.TestCloud.Company.arrowhead.eu",
  "profile": "SYSTEM",
  "serialNumber": "4189327123456789",
  "validAfter": "2026-10-22T12:00:00Z",
  "validBefore": "2026-10-22T12:00:00Z"
}
```

Listing 2: An sign response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.3, which are used to represent things like hashes, identifiers and texts.

3.1 struct CertificateSigningRequest

Object Field	Value Type	Description
"encodedCSR"	String	PEM PKCS\#10 request or Base64 DER (alias `csr`).
"profile"	String	`on`, `de`, `sy` or `op`; default the CSR dnQualifier, else `sy`. Must be allowed for the caller.
"addresses"	Array<String>	DNS names and IP addresses for the subject alternative name (at most 16); default those of the CSR.
"validAfter"	DateTime	Start of validity, ISO 8601; default now minus one minute.
"validBefore"	DateTime	End of validity; default start plus the profile maximum (24 h for `on`, 365 days otherwise), never beyond the CA certificate.

3.2 struct CertificateSigningResult

Object Field	Value Type	Description
"id"	Number	Record identifier in the certificate register.
"certificateChain"	Array<String>	Issued certificate and CA certificate, Base64 DER, leaf first.
"commonName"	String	Subject common name.
"profile"	String	Granted profile: `ON_BOARDING`, `DEVICE`, `SYSTEM` or `OPERATOR`.
"serialNumber"	String	Serial number, decimal.
"validAfter"	DateTime	Start of validity.
"validBefore"	DateTime	End of validity.

3.3 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “certificateSigning – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateSigning-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

certificateValidation HTTP/TLS/JSON

Interface Design Description

Service ID: *"certificateValidation"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `certificateValidation` service of the CertificateAuthority system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation check-certificate . . . 4
2.2 operation check-trusted-key . . . 4
2.3 operation get-crl . . . 5

3 Data Models **6**
3.1 struct CertificateCheckRequest . . . 6
3.2 struct CertificateCheckResult . . . 6
3.3 struct TrustedKeyCheckRequest . . . 6
3.4 struct TrustedKeyCheckResult . . . 6
3.5 struct TrustedKeyRecord . . . 6
3.6 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `certificateValidation` service interface of the CertificateAuthority system, which provides checking certificates and trusted keys against the CA's register, and publishing the certificate revocation list.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `certificateValidation` service interface

This document provides the Interface Design Description IDD to the *certificateValidation – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `certificateValidation` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `certificate-authority`, port 8448 and base path `/certificate-authority`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`check-certificate`	POST	`/checkCertificate`
`check-trusted-key`	POST	`/checkTrustedKey`
`get-crl`	GET	`/crl`

2 Service Operations

The interfaces of the `certificateValidation` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `certificateValidation` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 POST /certificate-authority/checkCertificate

Operation: check-certificate
Input: CertificateCheckRequest
Output: CertificateCheckResult

Returns the status of a certificate.

A successful call returns `200 OK` with the output in the body.

```
POST /certificate-authority/checkCertificate HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "certificate": "<PEM or Base64 DER>"
}
```

Listing 1: An check-certificate invocation.

```
{
  "status": "ok",
  "valid": true,
  "serialNumber": "4189327123456789",
  "known": true
}
```

Listing 2: An check-certificate response.

2.2 POST /certificate-authority/checkTrustedKey

Operation: check-trusted-key
Input: TrustedKeyCheckRequest
Output: TrustedKeyCheckResult

Returns whether a public key is trusted now.

A successful call returns `200 OK` with the output in the body.

```
POST /certificate-authority/checkTrustedKey HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
```

```
  "publicKey": "-----BEGIN PUBLIC KEY-----\\n..."
}
```

Listing 3: An check-trusted-key invocation.

```
{
  "trusted": true,
  "key": {
    "id": 42,
    "public_key_pem": "-----BEGIN PUBLIC KEY-----\\n...",
    "description": "Display may read",
    "valid_after": "string",
    "valid_before": "string",
    "created_at": "string"
  }
}
```

Listing 4: An check-trusted-key response.

2.3 GET /certificate-authority/crl

Operation: get-crl

Returns the current CRL (every revoked, unexpired certificate; next update after `CRL_VALIDITY_HOURS`, 168).
Returns the current X.509 v2 CRL, PEM, `application/pkix-crl`.
A successful call returns `200 OK`.

```
GET /certificate-authority/crl HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An get-crl invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct CertificateCheckRequest

Object Field	Value Type	Description
"certificate"	String	Certificate to check, PEM or Base64 DER (alias `encodedCertificate`).

3.2 struct CertificateCheckResult

Object Field	Value Type	Description
"status"	String	`GOOD`, `REVOKED`, `EXPIRED` or `UNKNOWN` (not issued by this CA).
"valid"	Boolean	True only for `GOOD`.
"serialNumber"	String	Serial number, decimal.
"known"	Boolean	Whether the serial is in the register (only for certificates of this CA).

3.3 struct TrustedKeyCheckRequest

Object Field	Value Type	Description
"publicKey"	String	PEM SubjectPublicKeyInfo (alias `publicKeyPem`).

3.4 struct TrustedKeyCheckResult

Object Field	Value Type	Description
"trusted"	Boolean	Whether the key is a trusted key within its validity.
"key"	TrustedKeyRecord	The matching record, or null.

3.5 struct TrustedKeyRecord

Object Field	Value Type	Description
"id"	Number	Record identifier.
"public_key_pem"	String	The key, PEM.
"description"	String	Description.
"valid_after"	String	Start of validity.
"valid_before"	String	End of validity.
"created_at"	String	Creation time.

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “certificateValidation – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateValidation-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the CertificateAuthority system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation echo . . . 4
2.2 operation power . . . 4
2.3 operation status . . . 4

3 Data Models **6**
3.1 struct Observation . . . 6
3.2 Primitives . . . 6

4 References **7**

5 Revision History **8**
5.1 Amendments . . . 8
5.2 Quality Assurance . . . 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of CertificateAuthority.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, CertificateAuthority registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.CertificateAuthority.CertificateAuthority",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher is the `Messaging.Publisher` of this independently packaged CertificateAuthority system, configured with `AMQP_HOST`, `AMQP_PORT`, `LOCAL_CLOUD` and `MONITOR_INTERVAL_SECONDS` (30); its `echo` value is `status=alive` and the metadata add `implementation=java`. The publisher connects to the broker (default `amqp-broker`, port 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 10 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.CertificateAuthority.CertificateAuthority.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.CertificateAuthority.CertificateAuthority.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.CertificateAuthority.CertificateAuthority.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "CertificateAuthority-echo-1792670400000-42",
  "providerSystem": "CertificateAuthority",
  "serviceInstance": "CertificateAuthority",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "CertificateAuthority is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
```

```
17     }
18 }
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power`, `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

HTTPS, mandatory mutual TLS, Arrowhead identity-token header and JSON request/response documents. General management is sysop/whitelist protected; enrollment endpoints use approved certificate fingerprints and profile/serial checks instead of ordinary identity tokens. Error responses contain errorMessage, errorCode, origin and timestamp. Exact DTO and status/error parity is not certified.

1.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
`POST`	`/certificate-authority/sign`	identity
`POST`	`/certificate-authority/checkCertificate`	identity
`GET`	`/certificate-authority/crl`	identity
`GET`	`/certificate-authority/mgmt/certificates`	management
`DELETE`	`/certificate-authority/mgmt/certificates/{id}`	identity
`GET`	`/certificate-authority/mgmt/keys`	management
`DELETE`	`/certificate-authority/mgmt/keys/{id}`	identity
`POST`	`/certificate-authority/mgmt/revocations`	management
`PUT`	`/certificate-authority/mgmt/keys`	management
`POST`	`/certificate-authority/checkTrustedKey`	identity
`GET`	`/certificate-authority/monitor/echo`	certificate
`GET`	`/certificate-authority/general/mgmt/get-config`	management
`POST`	`/certificate-authority/general/mgmt/logs`	management

1.2 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- CSR and CRL operations use a separately provisioned CA key; no CA key is generated/copied by this port. CRL export does not prove enforcement by every relying party. Java SQL import, automatic CRL file publication, HSM/OCSP/HA are absent.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

1 CertificateAuthority

1.1 System overview

Native Rust development profile for local Java source `JavaSupportRuntime/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Native CSR verification/signing, profiles, revocation, PEM CRLs, trusted keys

1.5 Open release gates

Explicit SANs only (no CSR extension copying); CRL file publication/reload, OCSP/HSM/HA, full validity/DTO normalization. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `certificateAuthorityManagement`, `certificateSigning`, `certificateValidation`, `echo`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/certificate-authority/general/mgmt/get-config	management
POST	/certificate-authority/general/mgmt/logs	management
GET	/certificate-authority/monitor/echo	certificate
POST	/certificate-authority/sign	certificate
GET	/certificate-authority/crl	certificate
POST	/certificate-authority/checkCertificate	certificate
POST	/certificate-authority/checkTrustedKey	certificate
GET	/certificate-authority/mgmt/certificates	certificate
DELETE	/certificate-authority/mgmt/certificates/{id}	certificate
POST	/certificate-authority/mgmt/revocations	certificate
GET	/certificate-authority/mgmt/keys	certificate
PUT	/certificate-authority/mgmt/keys	certificate
DELETE	/certificate-authority/mgmt/keys/{id}	certificate

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.